

# TP 5 : Apprentissage supervisé et KNN

## Optimisation du classifieur Titanic

Lola Etiévant

2026-03-02

### Table des matières

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>1. Préparation des Données</b>                     | <b>1</b> |
| 1.1      | 1.1 Chargement et Nettoyage . . . . .                 | 2        |
| 1.2      | 1.2 Transformation des Variables (Encoding) . . . . . | 2        |
| 1.2.1    | Dummy Coding . . . . .                                | 2        |
| 1.2.2    | Label Encoding . . . . .                              | 2        |
| 1.3      | 1.3 Dataframe Prédictif Final . . . . .               | 3        |
| <b>2</b> | <b>2. Stratégie de Validation (Holdout 80/10/10)</b>  | <b>3</b> |
| 2.1      | 2.1 Partitionnement . . . . .                         | 3        |
| 2.2      | 2.2 Pré-traitement (Éviter le Data Leakage) . . . . . | 4        |
| <b>3</b> | <b>3. Sélection de l'Hyperparamètre</b>               | <b>4</b> |
| <b>4</b> | <b>4. Validation Croisée (10-Fold CV)</b>             | <b>5</b> |
| <b>5</b> | <b>5. Évaluation Finale</b>                           | <b>6</b> |

## 1 1. Préparation des Données

### Objectif

Préparer les variables pour qu'elles soient compatibles avec le calcul de distance Euclidienne utilisé par le KNN (K-Nearest Neighbors).

## 1.1 1.1 Chargement et Nettoyage

```
# Chargement des fichiers
qualitative_vars <- read.csv("../TP2/titanic_pre_processed_qualitative_vars.csv")
quantitative_vars <- read.csv("../TP2/titanic_pre_processed_quantitative_vars.csv")
target_df <- read.csv("../TP2/titanic_pre_processed_target.csv", stringsAsFactors = F)

# Conversion immédiate en vecteur de facteurs (plus robuste que le dataframe)
target <- as.factor(target_df$x)

# Suppression des valeurs manquantes pour 'Sex' (impact négligeable < 0.2%)
isMissingSex <- which(is.na(qualitative_vars$Sex) | qualitative_vars$Sex == "")
if (length(isMissingSex) > 0) {
  qualitative_vars <- qualitative_vars[-isMissingSex, ]
  quantitative_vars <- quantitative_vars[-isMissingSex, ]
  target <- target[-isMissingSex]
}
```

## 1.2 1.2 Transformation des Variables (Encoding)

### 1.2.1 Dummy Coding

On transforme les variables catégorielles en colonnes binaires (0/1).

```
make_dummies <- function(var_name, data) {
  encoder <- dummyVars(paste0("~", var_name), data = data)
  as.data.frame(predict(encoder, newdata = data))
}

qualitative_vars_dummy <- data.frame(
  make_dummies("Sex", qualitative_vars),
  make_dummies("withFam", qualitative_vars)
)
```

### 1.2.2 Label Encoding

Pour `Pclass`, nous utilisons une valeur numérique car il existe une notion d'ordre (1ère > 2ème > 3ème classe).

```
qualitative_vars_encoded <- data.frame(
  Pclass = as.numeric(as.character(qualitative_vars$Pclass))
)
```

### 1.3 1.3 Dataframe Prédicatif Final

```
predictive_vars <- data.frame(qualitative_vars_encoded, qualitative_vars_dummy, quantitative_vars)

# Suppression des redondances (Sex.female est l'inverse exact de Sex.male)
predictive_vars$Sex.male <- NULL
predictive_vars$withFam.0 <- NULL
predictive_vars$SibSp <- NULL
predictive_vars$Parch <- NULL
```

## 2 2. Stratégie de Validation (Holdout 80/10/10)

### 2.1 2.1 Partitionnement

Nous divisons les données pour éviter le **surapprentissage** et garantir une évaluation honnête.

```
set.seed(1234)
n <- nrow(predictive_vars)

# 1. Isoler le Test final (10%)
inTest <- sample(1:n, size = round(0.1 * n))
notinTest <- (1:n)[-inTest]

# 2. Isoler la Validation (10%)
inValidation <- sample(notinTest, size = round(0.1 * n))

# 3. Création des jeux de données
X_test <- predictive_vars[inTest,]
y_test <- target[inTest]

X_val <- predictive_vars[inValidation,]
y_val <- target[inValidation]

X_train <- predictive_vars[-c(inTest, inValidation),]
y_train <- target[-c(inTest, inValidation)]
```

## 2.2 2.2 Pré-traitement (Éviter le Data Leakage)

### ⚠ Important

L'imputation et la normalisation doivent être calculées sur le **Train** et appliquées au **Validation/Test**.

```
# Calcul des paramètres sur le Train uniquement
Proc <- preProcess(X_train, method = c("medianImpute", "range"))

# Application
X_train_scaled <- predict(Proc, X_train)
X_val_scaled   <- predict(Proc, X_val)
X_test_scaled  <- predict(Proc, X_test)
```

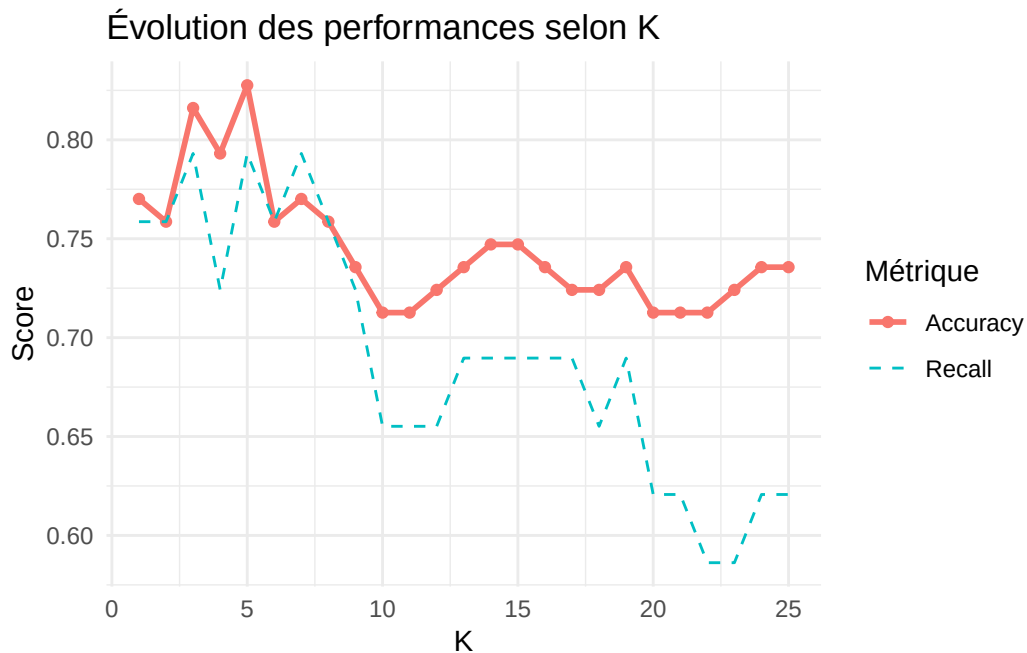
## 3 3. Sélection de l'Hyperparamètre

```
KGrid <- 1:25
metrics <- data.frame(K = KGrid, Accuracy = 0, Recall = 0)

for(i in seq_along(KGrid)){
  set.seed(1234)
  pred <- knn(train = X_train_scaled, test = X_val_scaled, cl = y_train, k = KGrid[i])
  cm   <- confusionMatrix(pred, y_val, positive = "Survivant")

  metrics$Accuracy[i] <- cm$overall["Accuracy"]
  metrics$Recall[i]   <- cm$byClass["Sensitivity"]
}

# Visualisation graphique
ggplot(metrics, aes(x = K)) +
  geom_line(aes(y = Accuracy, color = "Accuracy"), size = 1) +
  geom_point(aes(y = Accuracy, color = "Accuracy")) +
  geom_line(aes(y = Recall, color = "Recall"), linetype = "dashed") +
  theme_minimal() +
  labs(title = "Évolution des performances selon K", y = "Score", color = "Métrique")
```



## 4 4. Validation Croisée (10-Fold CV)

### 💡 Pourquoi la CV ?

La validation croisée réduit l'aléa lié au découpage des données en moyennant les performances sur plusieurs plis.

```
X_remain <- predictive_vars[-inTest,]
y_remain <- target[-inTest]

nfolds <- 10
folds <- sample(1:nfolds, nrow(X_remain), replace = TRUE)
cv_results <- matrix(NA, nrow = nfolds, ncol = length(KGrid))

for(f in 1:nfolds){
  idx_val <- which(folds == f)

  # Split et Scale local (Interne au pli)
  X_tr_cv <- X_remain[-idx_val,]; X_va_cv <- X_remain[idx_val,]
  y_tr_cv <- y_remain[-idx_val]; y_va_cv <- y_remain[idx_val]
```

```

proc_cv <- preProcess(X_tr_cv, method = c("medianImpute", "range"))
X_tr_cv_s <- predict(proc_cv, X_tr_cv)
X_va_cv_s <- predict(proc_cv, X_va_cv)

for(k in seq_along(KGrid)){
  pred_cv <- knn(X_tr_cv_s, X_va_cv_s, y_tr_cv, k = KGrid[k])
  cv_results[f, k] <- mean(pred_cv == y_va_cv)
}
}

mean_acc <- colMeans(cv_results)
best_k <- KGrid[which.max(mean_acc)]

```

L'hyperparamètre optimal sélectionné par Validation Croisée est  $k = 7$ .

## 5 5. Évaluation Finale

Nous évaluons maintenant notre modèle “final” (entraîné sur Train+Validation) sur le jeu de **Test** que le modèle n’a jamais vu.

```

# Pré-traitement final
Proc_final <- preProcess(X_remain, method = c("medianImpute", "range"))
X_remain_scaled <- predict(Proc_final, X_remain)
X_test_scaled <- predict(Proc_final, X_test)

# Prédiction
final_pred <- knn(X_remain_scaled, X_test_scaled, y_remain, k = best_k)

# Matrice de Confusion
final_cm <- confusionMatrix(final_pred, y_test, positive = "Survivant")
final_cm

```

Confusion Matrix and Statistics

|            | Reference |           |
|------------|-----------|-----------|
| Prediction | Décédé    | Survivant |
| Décédé     | 43        | 11        |
| Survivant  | 5         | 28        |

Accuracy : 0.8161

95% CI : (0.7186, 0.8911)  
No Information Rate : 0.5517  
P-Value [Acc > NIR] : 1.973e-07

Kappa : 0.6228

McNemar's Test P-Value : 0.2113

Sensitivity : 0.7179  
Specificity : 0.8958  
Pos Pred Value : 0.8485  
Neg Pred Value : 0.7963  
Prevalence : 0.4483  
Detection Rate : 0.3218  
Detection Prevalence : 0.3793  
Balanced Accuracy : 0.8069

'Positive' Class : Survivant

#### ! Conclusion

Le modèle final présente une précision de **81.61%** sur des données inconnues.